



Cité des Métiers et des Compétences – CMC Tanger

Filière IDOSR

End of Studies Project Report

Supervision and monitoring of network and server infrastructure at Disway

Student: Marouane AABIRROUCHE

Host Company: Disway

Professional Supervisor: Youness MOHIB

Academic Year 2025–2026



Acknowledgements

I would like to express my deep gratitude to the Cité des Métiers et des Compétences (CMC Tanger) for the quality of its teaching and continuous support.

I thank the management and the IT team at Disway for their welcome, technical feedback, and the operational access provided to the lab.

A special thanks to my professional supervisor for their guidance throughout this internship.

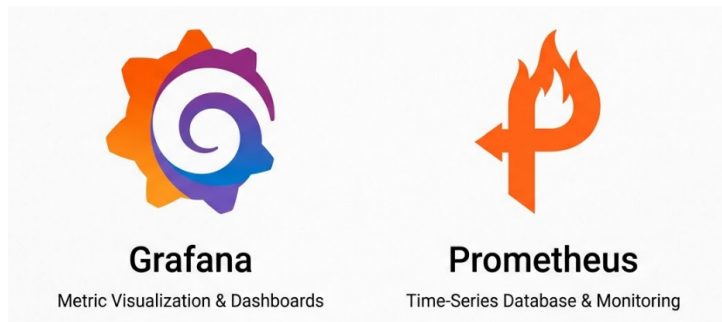
Contents

Acknowledgements	1
General Introduction	3
Chapter 1 – Company and Project Context	4
Chapter 2 – Theory and Technical Choices	5
Chapter 3 – Practical Engineering and Installation	6
0.1 Topology and Bridged Mode	6
0.2 Static Network Configuration	6
0.3 Installation and Configuration of Exporters as systemd Services	6
0.4 Windows Defender Firewall: Opening and Restricting Port 9182	7
0.5 Advanced Relabeling in <code>prometheus.yml</code>	7
0.6 Minimal Network Security	8
Chapter 4 – Results, PromQL, and Grafana Dashboards	9
0.7 Bandwidth: Principles and PromQL Query	9
0.8 Port Status and Visual Mapping	9
0.9 Server Supervision: CPU and Memory	10
0.10 Blackbox: Application Probes (Jellyfin)	10
0.11 Alerting and Workflow	11
0.12 E-mail Alerting	11
Final Result: Supervision Interface	13
Internship Review and General Conclusion	14
Technical Appendices	15

General Introduction

Infrastructure supervision is a critical function for any organization heavily dependent on IT systems and networks. At Disway, the SAPINO logistics platform and associated services orchestrate sensitive operations whose continuity conditions commercial activity.

This report presents the transition to proactive supervision centered on Prometheus and Grafana, covering Linux, Windows, network infrastructure via SNMP, and Blackbox application probes.



Chapter 1 – Company and Project Context

Disway is a major distribution player operating in Morocco, Tunisia, and Congo. Beyond its historical status as a leader in IT distribution, the group has significantly diversified through strategic subsidiaries such as **Solarway**, dedicated to the distribution of photovoltaic solar solutions, and **Disway Logistic Services (DLS)**. The latter orchestrates the immense SAPINO logistics platform, coordinating massive flows of goods, inventory tracking, and billing. Critical applications, notably the WMS (Warehouse Management System) and the B2B portal, are at the heart of these operations: prolonged downtime immediately impacts order fulfillment, delivery, and customer relations.

For these reasons, high availability, incident traceability, and the ability to diagnose rapidly are absolute business requirements. The previous manual approach allowed neither easy data correlation nor trend analysis, prompting the IT team to express the need for a centralized tool offering a “Single Pane of Glass”.

Technical constraints included integration with Windows and Linux environments, SNMP collection from HPE switches, and the ability to run active HTTP/TCP/ICMP probes to validate the overall application experience.



Chapter 2 – Theory and Technical Choices

The choice of Prometheus was obvious for its “Pull” model and optimized TSDB. The Pull model centralizes the scraping clock and facilitates diagnostics, while the TSDB allows for rapid time-series queries and efficient compaction.

Four exporters were deployed to cover the entire fleet: Node Exporter for Linux, Windows Exporter for Microsoft servers, SNMP Exporter for HPE switches, and Blackbox Exporter for active probes. This combination provides system, network, and application metrics, enabling data correlation.

Node Exporter exposes kernel and OS metrics (CPU, memory, disk I/O, network interfaces).

Windows Exporter queries WMI/PerfMon and exposes CPU, memory, and I/O usage for Windows servers. SNMP Exporter acts as an SNMP->Prometheus proxy, notably retrieving ifHCInOctets/ifHCOctets and ifOperStatus. Blackbox Exporter performs HTTP/TCP/ICMP probes to validate service availability and latency.

Table 1: Synthetic Comparison: Prometheus vs Zabbix

Criterion	Prometheus	Zabbix
Collection Model	Pull (scraping exporters): centralized and iterative.	Agents / Push / SNMP: flexible but often more intrusive.
Database	Built-in TSDB, optimized for time series.	Relational database (MySQL/-Postgres): robust but maintenance-heavy.
Learning Curve	Relatively fast for basic concepts; rich modular ecosystem.	Longer: templates, items, and triggers require deep learning.
Configuration Complexity	Modular and config-as-code oriented (powerful relabeling).	Powerful but centralized and heavier to configure in large deployments.

Zabbix remains a highly powerful solution, but its operational weight and deployment complexity explain the choice of Prometheus for this lab project, where flexibility, native TSDB, and ease of prototyping took precedence.

Chapter 3 – Practical Engineering and Installation

This chapter describes the topology, machine configuration, commissioning of exporters as services, firewall rules, and advanced Prometheus configuration (relabeling).

0.1 Topology and Bridged Mode

The VMs were configured in Bridged mode on VMware to coexist on the same VLAN as the physical HPE switch. The Prometheus server received the address 192.168.174.137, the Windows VM 192.168.174.136, the Linux VM 192.168.174.138, and the HPE switch 192.168.174.200. This topology facilitates SNMP polling and application testing.

As illustrated in Figure 1, the Prometheus server centralizes calls to local exporters and proxies requests to physical targets.

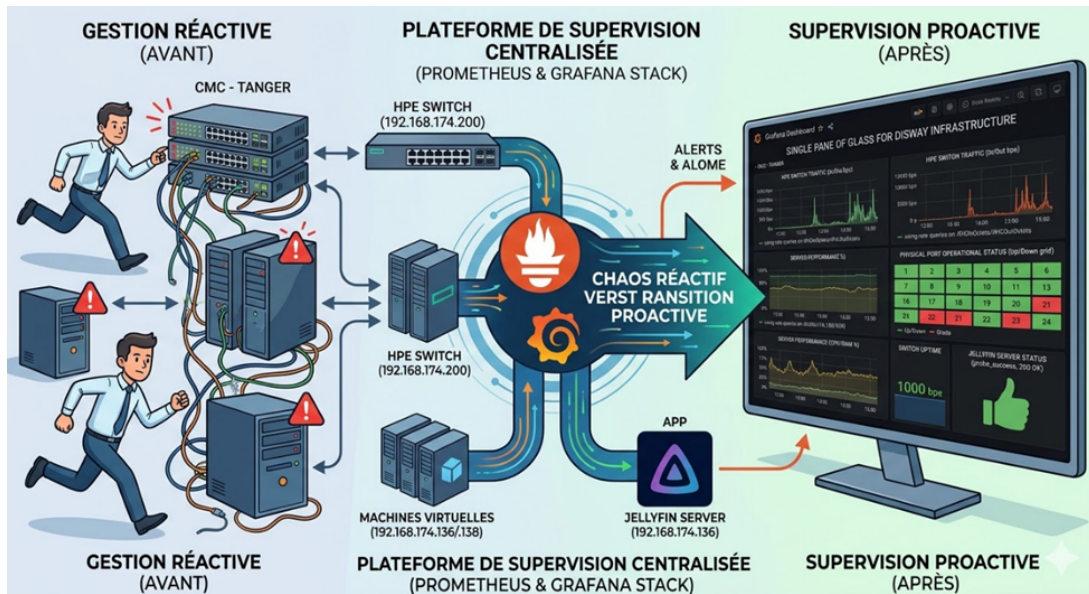


Figure 1: Laboratory network topology and transition to centralized supervision

0.2 Static Network Configuration

On Ubuntu, Netplan was used to configure static addresses. The example YAML file for the `ens33` interface contains the address, gateway, and DNS servers, and is applied with `sudo netplan apply`.

On Windows, addresses are configured via the network interface or PowerShell to ensure consistency in the addressing plan.

0.3 Installation and Configuration of Exporters as systemd Services

For production, each exporter was transformed into a systemd service. The deployment follows these principles: binaries in `/usr/local/bin`, system users without a shell, unit files under `/etc/systemd/system`, systemd reload, automated enable and start.

The advantages are multiple: automatic restart in case of a crash, centralized logs, control via `systemctl`, and integration into operational procedures.

Example of a systemd unit for `node_exporter`:

```

1 [Unit]
2 Description=Node Exporter
3 Wants=network-online.target
4 After=network-online.target
5
6 [Service]
7 User=node_exporter
8 Group=node_exporter
9 Type=simple
10 ExecStart=/usr/local/bin/node_exporter
11 Restart=on-failure
12 RestartSec=5s
13
14 [Install]
15 WantedBy=multi-user.target

```

Procedure to deploy a service: copy the binary, create the user, drop the `.service` file, run `sudo systemctl daemon-reload` and then `sudo systemctl enable -now <service>`.

0.4 Windows Defender Firewall: Opening and Restricting Port 9182

The Windows Exporter exposes a local HTTP endpoint on port 9182. An inbound rule is required to allow scraping from the Prometheus server. To limit exposure, the rule must be restricted to the source address of the Prometheus server:

```

1 New-NetFirewallRule -DisplayName "Prometheus_Windows_Exporter" -
  Direction Inbound -Action Allow -Protocol TCP -LocalPort 9182 -
  RemoteAddress 192.168.174.137

```

After applying, test the endpoint from Prometheus: `curl http://192.168.174.136:9182/metrics`.

0.5 Advanced Relabeling in `prometheus.yml`

The SNMP and Blackbox Exporters run locally. For Prometheus to address local exporters while retaining the targets' identities, relabeling is applied: the original target address is preserved in `__param_target`, and then `__address__` is replaced by `localhost:9116` (SNMP) or `localhost:9115` (Blackbox). The local exporter then uses the `target` value to poll the physical target.

Commented excerpt:

```

1 - job_name: 'snmp'
2   metrics_path: /snmp

```

```

3  params:
4      module: [if_mib]
5      static_configs:
6          - targets: ['192.168.174.200']
7      relabel_configs:
8          - source_labels: [__address__]
9            target_label: __param_target
10         - replacement: localhost:9116
11           target_label: __address__
12         - source_labels: [__param_target]
13           target_label: instance

```

This technique centralizes probes, simplifies ACLs, and preserves metric readability by keeping the `instance` label with the real IP.

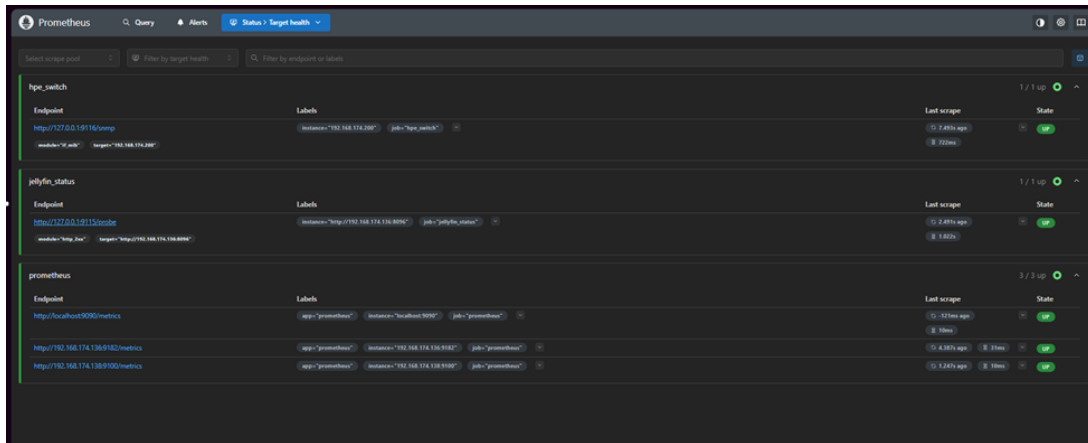


Figure 2: Prometheus interface confirming successful relabeling with targets in UP status

0.6 Minimal Network Security

On the HPE switch, an ACL was applied to only allow the Prometheus server IP (192.168.174.137) on the UDP/161 SNMP port. Management ports (SSH, HTTP management) are restricted to administration addresses.

Chapter 4 – Results, PromQL, and Grafana Dashboards

0.7 Bandwidth: Principles and PromQL Query

To measure throughput, the 64-bit ifHCInOctets/ifHCOutOctets counters are used. The standard query for inbound throughput in bps:

```
1 rate(ifHCInOctets{job="snmp", instance="192.168.174.200", ifDescr="
  GigabitEthernet1/0/1"}[5m]) * 8
```

The 5-minute window ensures appropriate smoothing; multiplying by 8 converts bytes to bits.

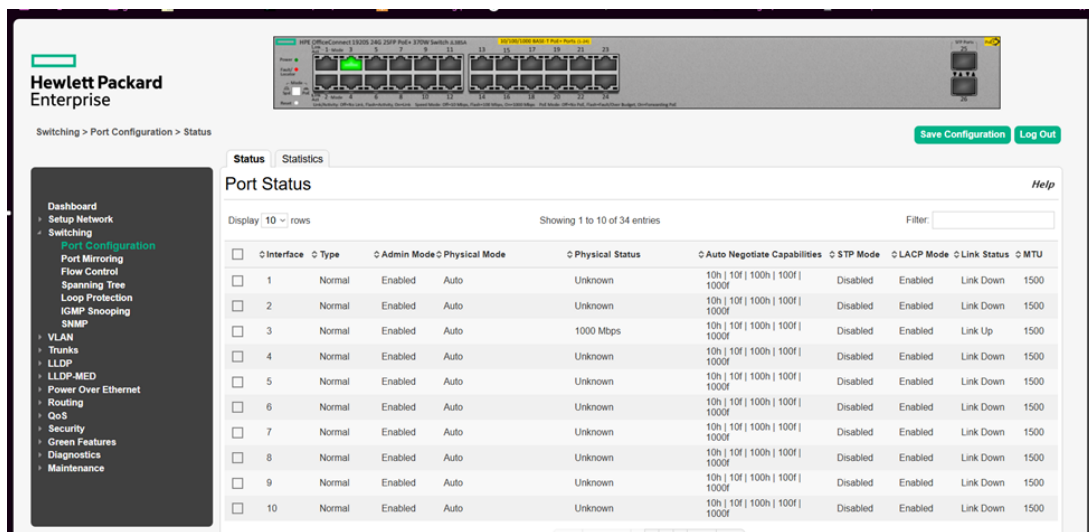


Figure 3: Evolution of inbound and outbound bandwidth of the HPE switch in bps

0.8 Port Status and Visual Mapping

The ifOperStatus metric returns 1 (up) or 2 (down). In Grafana, a Value Mapping transforms these values into colored indicators to facilitate operational intervention.

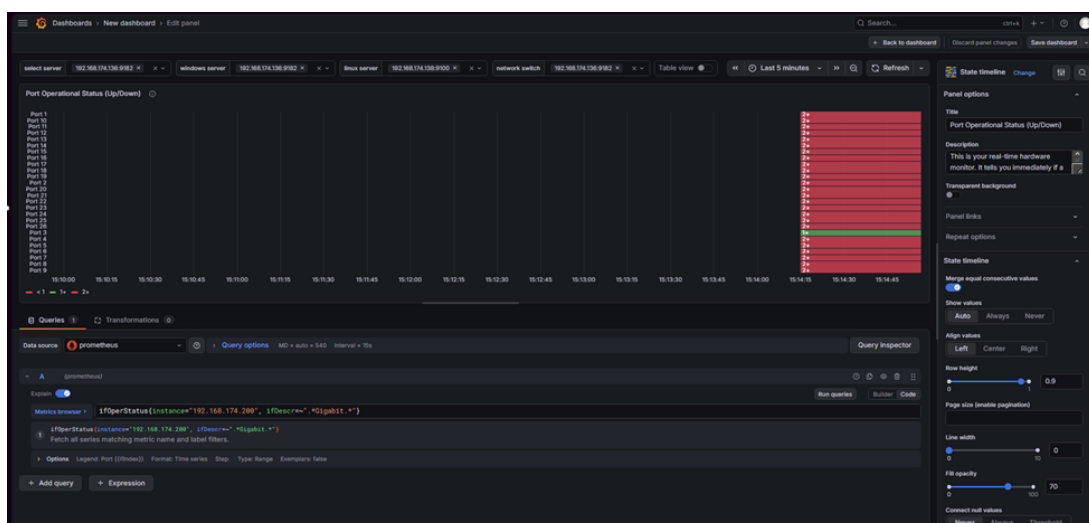


Figure 4: Visual translation of physical port status via Value Mappings

0.9 Server Supervision: CPU and Memory

CPU query used:

```
1 100 * (1 - avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

Operational thresholds: 80% warning, 90% critical (persisting for 5 minutes).

Memory query:

```
1 (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100
```

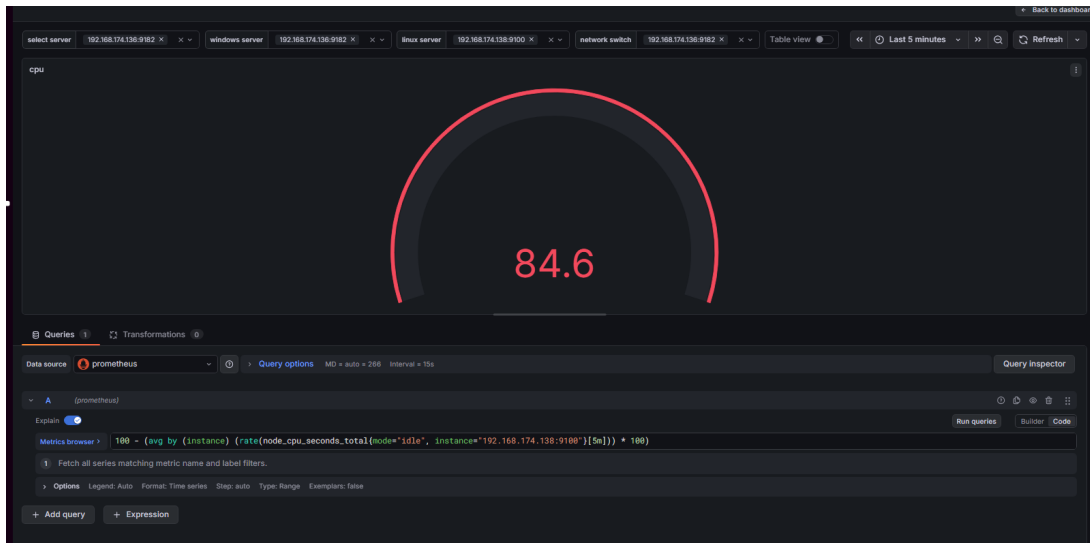


Figure 5: Supervision gauges for CPU and RAM allocation of virtual servers

0.10 Blackbox: Application Probes (Jellyfin)

The `probe_success` metric indicates a successful probe. Safety condition:

```
1 probe_success{job="blackbox", instance="http://192.168.174.150:8096"} == 1
```

Additional metrics (`probe_duration_seconds`, `probe_http_status_code`) help qualify degradations.

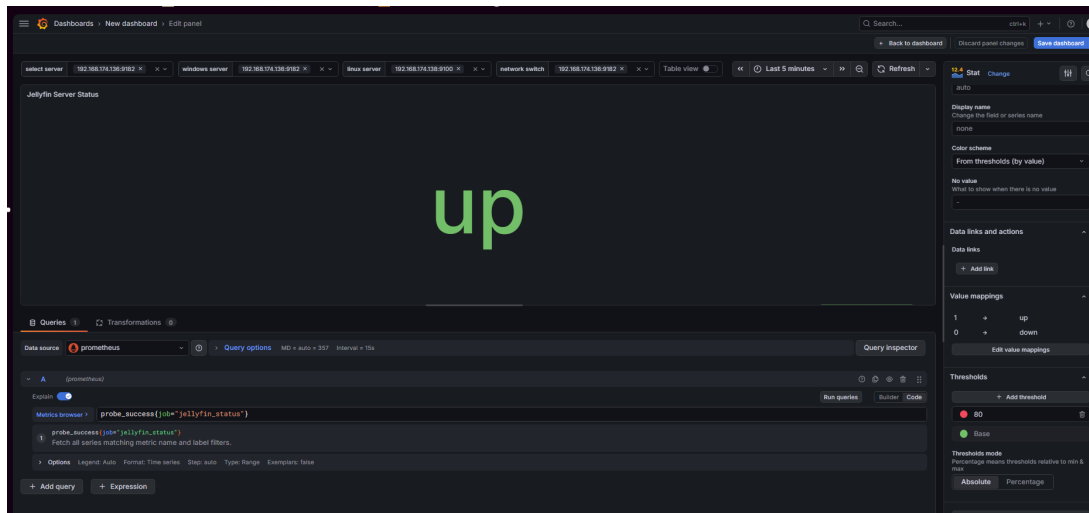


Figure 6: Synthetic Blackbox probe confirming application availability

0.11 Alerting and Workflow

Alert examples: CPU > 90% for 5 minutes; ifOperStatus == 2 for 1 minute; probe_success == 0 for 2 minutes. Notifications should include a short runbook and context (instance, timestamp, measured value).

0.12 E-mail Alerting

E-mail notification is a classic and reliable method to alert administrators. In our architecture, Grafana (or Alertmanager if using the native Prometheus chain) is configured to send e-mails via an SMTP relay. Below is an example of a simple alert and the expected SMTP contact.

Alert Example

```
1 groups:
2   - name: windows_rules
3     rules:
4       - alert: WindowsCPUOverload
5         expr: avg by (instance) (rate(windows_cpu_time_total[5m])) > 0.8
6         for: 2m
7         labels:
8           severity: critical
9         annotations:
10          summary: "High CPU on {{ $labels.instance }}"
11          description: "Average CPU load on {{ $labels.instance }}
           exceeds 80% for 2 minutes."
```

SMTP Configuration (example): Grafana/Alertmanager will use an SMTP contact with the following parameters:

- Server: smtp.example.com
- Port: 587

- Method: STARTTLS
- User: alert@disway.example
- Password: (via environment variable or secret manager)

Note: the proof image is intentionally omitted for compilation; the space below is reserved if needed.

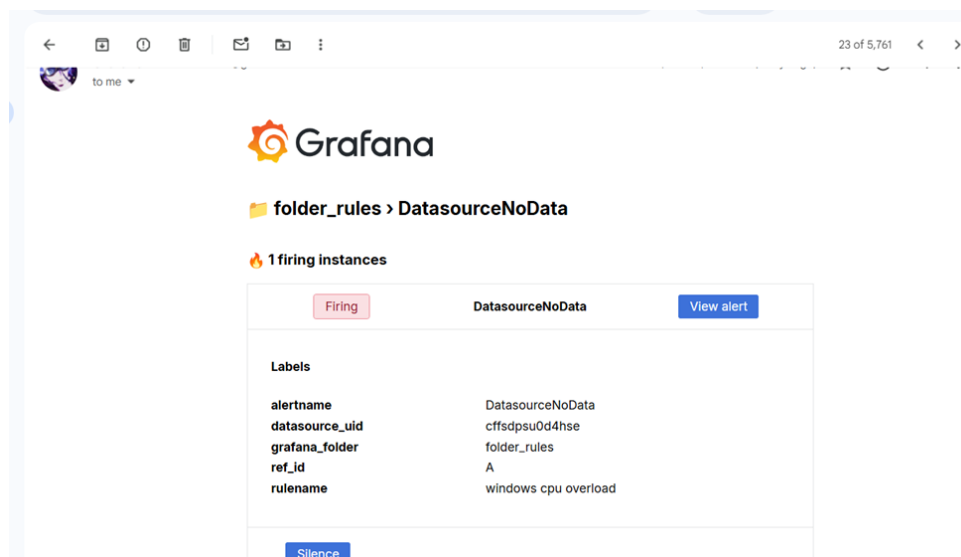


Figure 7: Example of received e-mail alert

Final Result: Supervision Interface

The deployed visualization platform (Grafana) gathers the main operational indicators on a single dashboard. The final result shows several panels coexisting to offer a comprehensive overview actionable by the team:

- Server indicators (Linux and Windows): gauges and graphs showing memory (RAM) usage, processor (CPU) utilization, and available disk space. These panels facilitate the rapid identification of degraded machines or those approaching critical thresholds.
- Service status: binary indicators (UP/DOWN) or resilience metrics to validate whether critical services (Jellyfin, application database, agents) are reachable and responding.
- Network consumption: input/output graphs per interface (bps) to visualize inbound/outbound traffic, load spikes, and the relative latency of Blackbox probes.
- Uptime and running duration: counters displaying the time (in seconds/hours) since the last restart, useful for correlating potential reboots with incidents.
- Switch Panel: specific views showing port utilization and mapping (wall jack → port).

The space below is reserved for a screenshot of the dashboard (e.g., Grafana capture showing CPU/RAM gauges, network graphs, statuses, and switch panels). Replace the image file with `images/dashboard_overview.png` or uncomment if you do not wish to attach the image.



Figure 8: Grafana dashboard overview: server, network, and switch measurements

These dashboards are the result of the integration between Prometheus (collection), the exporters (Node, Windows, SNMP, Blackbox), and Grafana (visualization). They allow for both early detection of anomalies and rapid diagnostics via multi-source correlation (e.g., simultaneous CPU spike with an increase in traffic on a switch port).

Internship Review and General Conclusion

This internship within the Disway IT Department was a total immersion into IT infrastructure engineering, validating cross-functional technical skills ranging from system administration to the design of supervision architectures.

In terms of system administration and deployment, I managed high-stakes enterprise system integrations (network configurations, Active Directory junction) and hardware fleet management. A critical intervention required restoring administrator access to a production ESXi hypervisor; this operation was successfully carried out by booting the server via a Live Ubuntu environment to rewrite the system configuration files.

My responsibilities also covered advanced, hands-on hardware support and network infrastructure. I executed a critical hardware upgrade on a high-availability server, conducting strict compatibility tests to stabilize the architecture at 16 GB. Locally, I orchestrated the migration of a network distribution node (replacing a 24-port switch with a 48-port one). This operation required a strict patching plan, validated by a cable tester, guaranteeing a 1:1 physical and logical mapping.

The technical culmination of this experience was the realization of my End of Studies Project. Being allocated a dedicated physical switch allowed me to design, configure, and deploy a complete observability stack (Prometheus and Grafana). This project required mastering software engineering (Relabeling, PromQL queries, systemd automation) and network integration (SNMP) to deliver a centralized and functional supervision solution.

The architecture deployed opens up clear evolutionary perspectives for the IT Department: the deployment of Alertmanager for multi-channel notification escalation (Telegram, SMS), the implementation of a Prometheus cluster to ensure High Availability (HA), and the extension of SNMP monitoring to Uninterruptible Power Supplies (UPS) and Wi-Fi access points.

Ultimately, this experience crystallized the transition from academic theory to proactive production engineering. It forged my technical autonomy and my ability to meet the high-availability requirements of a large-scale infrastructure.

Technical Appendices

systemd unit file: node_exporter

```
1 [Unit]
2 Description=Node Exporter
3 Wants=network-online.target
4 After=network-online.target
5
6 [Service]
7 User=node_exporter
8 Group=node_exporter
9 Type=simple
10 ExecStart=/usr/local/bin/node_exporter
11 Restart=on-failure
12 RestartSec=5s
13
14 [Install]
15 WantedBy=multi-user.target
```

Excerpt from prometheus.yml (SNMP and Blackbox with relabeling)

```
1 global:
2   scrape_interval: 15s
3   evaluation_interval: 15s
4
5 scrape_configs:
6   - job_name: 'prometheus'
7     static_configs:
8       - targets: ['localhost:9090']
9
10    - job_name: 'node'
11      static_configs:
12        - targets: ['192.168.174.138:9100']
13
14    - job_name: 'windows'
15      static_configs:
16        - targets: ['192.168.174.136:9182']
17
18    # SNMP proxied via local snmp_exporter (relabeling below)
19    - job_name: 'snmp'
20      metrics_path: /snmp
21      params:
22        module: [if_mib]
23      static_configs:
24        - targets: ['192.168.174.200']
25      relabel_configs:
26        - source_labels: [__address__]
27          target_label: __param_target
```

```
28     - replacement: localhost:9116
29       target_label: __address__
30     - source_labels: [__param_target]
31       target_label: instance
32
33   # Blackbox probes via local blackbox_exporter
34   - job_name: 'blackbox'
35     metrics_path: /probe
36     params:
37       module: [http_2xx]
38     static_configs:
39       - targets: ['http://192.168.174.150:8096']
40     relabel_configs:
41       - source_labels: [__address__]
42         target_label: __param_target
43       - replacement: localhost:9115
44         target_label: __address__
45       - source_labels: [__param_target]
46         target_label: instance
```

Windows Defender Command (restricted)

```
1 New-NetFirewallRule -DisplayName "Prometheus_Windows_Exporter" -
  Direction Inbound -Action Allow -Protocol TCP -LocalPort 9182 -
  RemoteAddress 192.168.174.137
```

Useful Commands

```
1 # Reload systemd
2 sudo systemctl daemon-reload
3
4 # Enable and start
5 sudo systemctl enable --now node_exporter
6
7 # Status
8 sudo systemctl status node_exporter
9
10 # Test Windows Exporter endpoint
11 curl http://192.168.174.136:9182/metrics
```

Technical References

Prometheus Documentation: <https://prometheus.io/docs>.

SNMP Exporter: https://github.com/prometheus/snmp_exporter.

Blackbox Exporter: https://github.com/prometheus/blackbox_exporter.

Windows Exporter: https://github.com/prometheus-community/windows_exporter.